

התאמת מחרוזות וקונבולוציה

חזרה קצרה על KMP | הגדרת קונבולוציה | תרגילים

התאמת מחרוזות

הגדרת הבעיה

נתון טקסט T במערך $T[1...n]$ שגודלו n ונתונה תבנית P במערך $P[1...m]$ שגודלה m .
אומרים ש- P מופיעה ב- T בהזזה של s אם $0 \leq s \leq n - m$ וכן $P[1...m] = T[s+1...s+m]$.

אם P מופיעה ב- T בהזזה s , אז s נקראת הזזה חוקית (valid shift), אחרת s היא הזזה לא חוקית.

המטרה

למצוא את כל ההזזות החוקיות של P בתוך T .

התאמת מחרוזות – אלגוריתם KMP

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

T=	a	b	a	b	b	a	b	b	a
P=	a	b	b	a					

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0			
---	--	--	--

$m = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0			
---	--	--	--

$m = 4$

$k = 0$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0			
---	--	--	--

$m = 4$

$k = 0$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0		
---	---	--	--

$m = 4$

$k = 0$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0		
---	---	--	--

$m = 4$

$k = 0$

$q = 3$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0		
---	---	--	--

$m = 4$

$k = 0$

$q = 3$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	
---	---	---	--

$m = 4$

$k = 0$

$q = 3$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	
---	---	---	--

$m = 4$

$k = 0$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	
---	---	---	--

$m = 4$

$k = 0$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	
---	---	---	--

$m = 4$

$k = 1$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$m = 4$

$k = 1$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$m = 4$

$k = 1$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 1$

$q = 0$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 1$

$q = 0$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 1$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 1$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 2$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 2$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 2$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 3$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 3$

$q = 0$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 3$

$q = 0$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 3$

$q = 0$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 3$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 4$

$q = 1$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 4$

$q = 2$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 5$

$q = 3$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 6$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print "Pattern occurs with shift"  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 6$

$q = 4$

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

```
1.  $m \leftarrow \text{length}[P]$ 
2.  $\pi[1] \leftarrow 0$ 
3.  $k \leftarrow 0$ 
4. for  $q \leftarrow 2$  to  $m$ 
5.     do while  $k > 0$  and  $P[k+1] \neq P[q]$ 
6.         do  $k \leftarrow \pi[k]$ 
7.     if  $P[k+1] = P[q]$ 
8.         then  $k \leftarrow k + 1$ 
9.      $\pi[q] \leftarrow k$ 
10. return  $\pi$ 
```

KMP-Matcher-Updated(T,P)

```
1.  $n \leftarrow \text{length}[T]$ 
2.  $m \leftarrow \text{length}[P]$ 
3.  $\pi \leftarrow \text{Compute-Prefix-Function}(P)$ 
4.  $q \leftarrow 0$ 
5. for  $i \leftarrow 1$  to  $n$ 
6.     do while  $q > 0$  and  $P[q+1] \neq T[i]$ 
7.         do  $q \leftarrow \pi[q]$ 
8.     if  $P[q+1] = T[i]$ 
9.         then  $q \leftarrow q + 1$ 
10.    if  $q = m$ 
11.        then print “Pattern occurs with shift”  $i - m$ 
12.         $q \leftarrow \pi[q]$ 
```

T=

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

P=

a	b	b	a
---	---	---	---

π =

0	0	0	1
---	---	---	---

$n = 9$

$m = 4$

$i = 6$

$q = 1$

“Pattern occurs with shift” 2

התאמת מחרוזות – אלגוריתם KMP

Compute-Prefix-Function(P)

1. $m \leftarrow \text{length}[P]$
2. $\pi[1] \leftarrow 0$
3. $k \leftarrow 0$
4. **for** $q \leftarrow 2$ **to** m
5. **do while** $k > 0$ and $P[k+1] \neq P[q]$
6. **do** $k \leftarrow \pi[k]$
7. **if** $P[k+1] = P[q]$
8. **then** $k \leftarrow k + 1$
9. $\pi[q] \leftarrow k$
10. **return** π

KMP-Matcher-Updated(T,P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $\pi \leftarrow \text{Compute-Prefix-Function}(P)$
4. $q \leftarrow 0$
5. **for** $i \leftarrow 1$ **to** n
6. **do while** $q > 0$ and $P[q+1] \neq T[i]$
7. **do** $q \leftarrow \pi[q]$
8. **if** $P[q+1] = T[i]$
9. **then** $q \leftarrow q + 1$
10. **if** $q = m$
11. **then** print “Pattern occurs with shift” $i - m$
12. $q \leftarrow \pi[q]$

$T =$

a	b	a	b	b	a	b	b	a
---	---	---	---	---	---	---	---	---

$P =$

a	b	b	a
---	---	---	---

$\pi =$

0	0	0	1
---	---	---	---

“Pattern occurs with shift” 2

“Pattern occurs with shift” 5

קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

$$c = a \otimes b \text{ סימון:}$$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

$$\text{כאשר: } c_j = \sum_{k=0}^j a_k \cdot b_{j-k} \text{ (סכום האינדקסים של הרכיבים שווה לאינדקס של } c \text{)}$$

לדוגמה:

$$\begin{aligned} a &= (a_0, a_1, a_2, \dots, a_{n-1}) \\ b &= (b_0, b_1, b_2, \dots, b_{n-1}) \\ c &= a \otimes b = ? \end{aligned}$$

קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

לדוגמה:

$$a = (a_0, a_1, a_2, \dots, a_{n-1})$$

$$b = (b_0, b_1, b_2, \dots, b_{n-1})$$

$$c = a \otimes b = (a_0 b_0, a_0 b_1 + a_1 b_0, a_0 b_2 + a_1 b_1 + a_2 b_0, \dots)$$

קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

$$c = a \otimes b \text{ סימון:}$$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

$$\text{כאשר: } c_j = \sum_{k=0}^j a_k \cdot b_{j-k} \text{ (סכום האינדקסים של הרכיבים שווה לאינדקס של } c \text{)}$$

לדוגמה:

$$a = (a_0, a_1, a_2, \dots, a_{n-1})$$

$$b = (b_0, b_1, b_2, \dots, b_{n-1})$$

$$c = a \otimes b = (a_0 b_0, a_0 b_1 + a_1 b_0, a_0 b_2 + a_1 b_1 + a_2 b_0, \dots)$$

$$j = 0, \quad 1, \quad 2, \quad \dots$$

קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.

קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

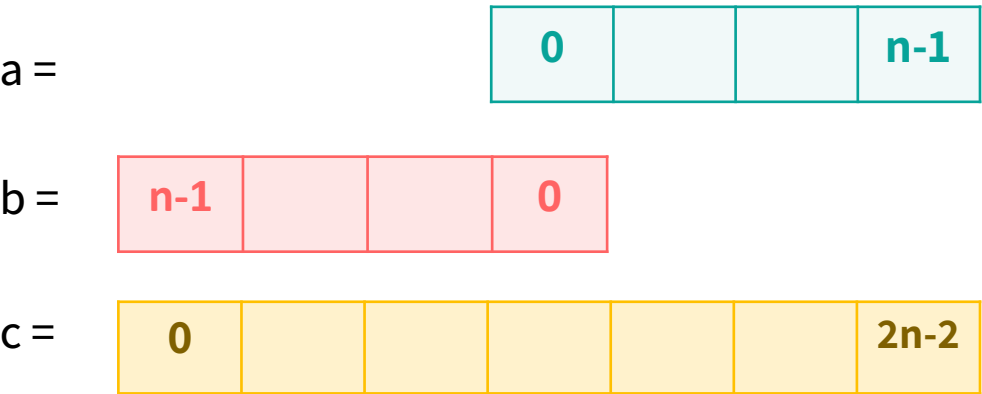
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

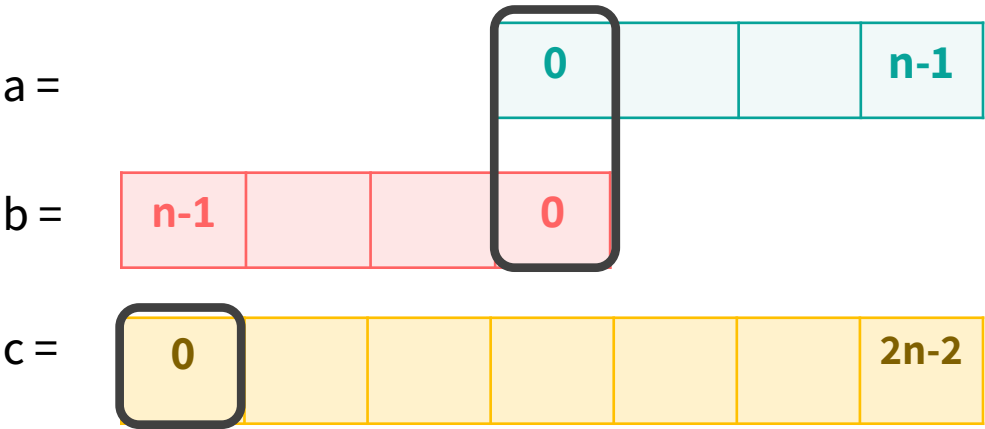
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

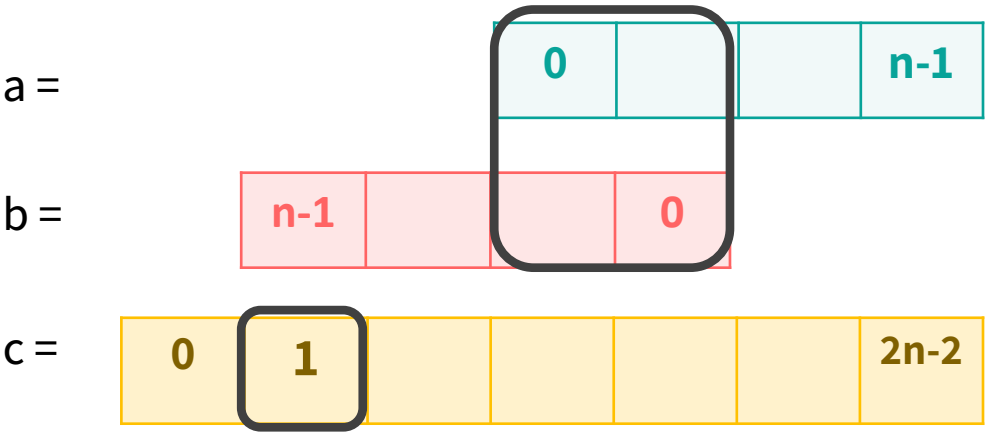
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

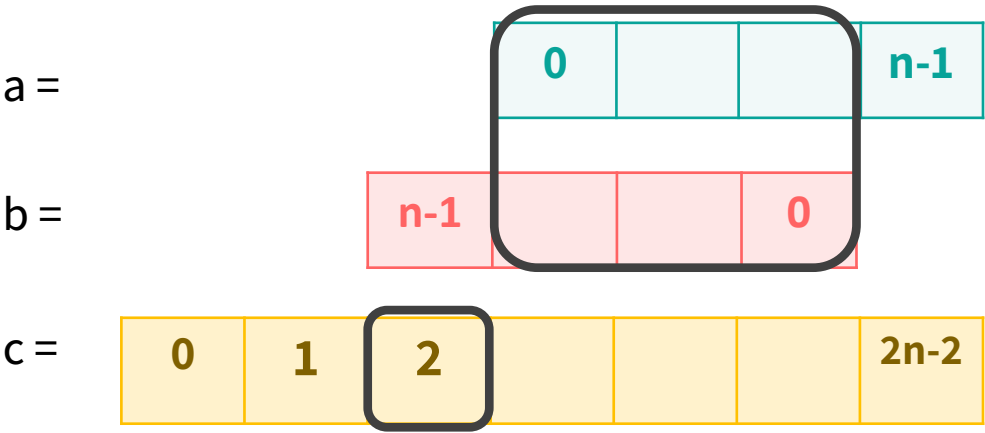
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

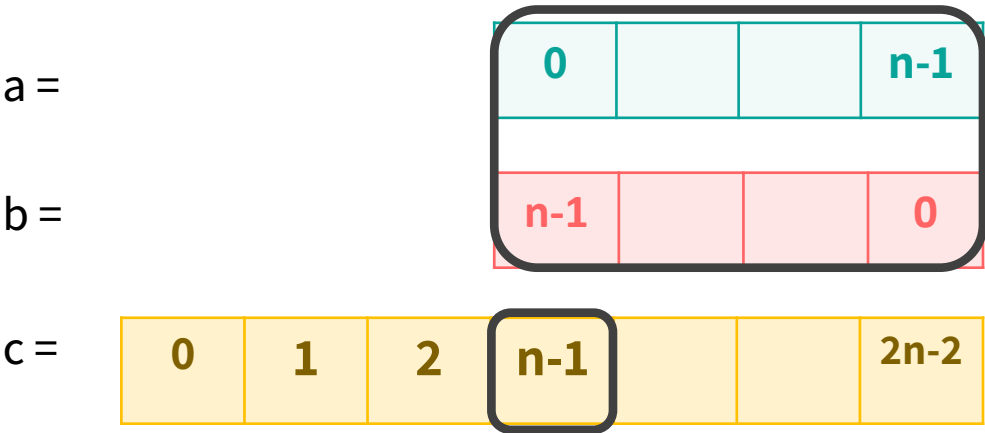
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

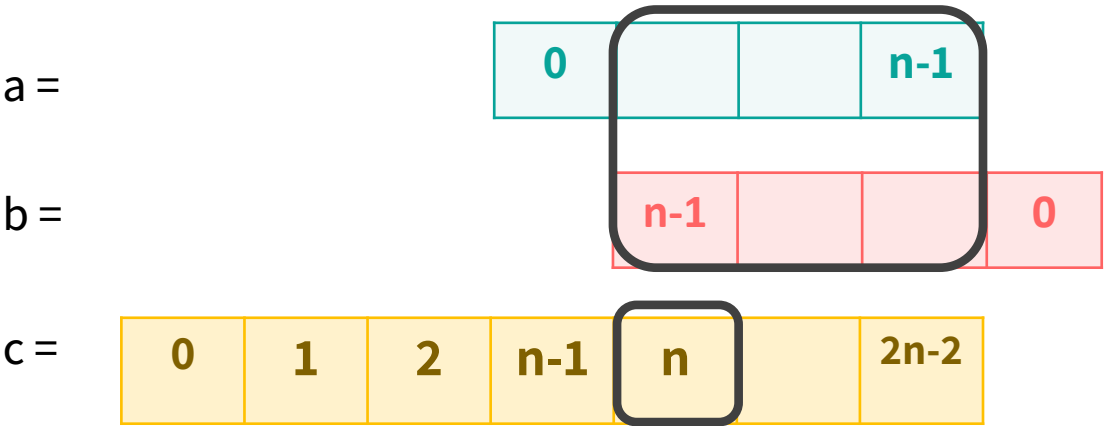
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



קונבולוציה - הגדרה

פעולת קונבולוציה בווקטורים: יהיו a, b וקטורים באורך n .

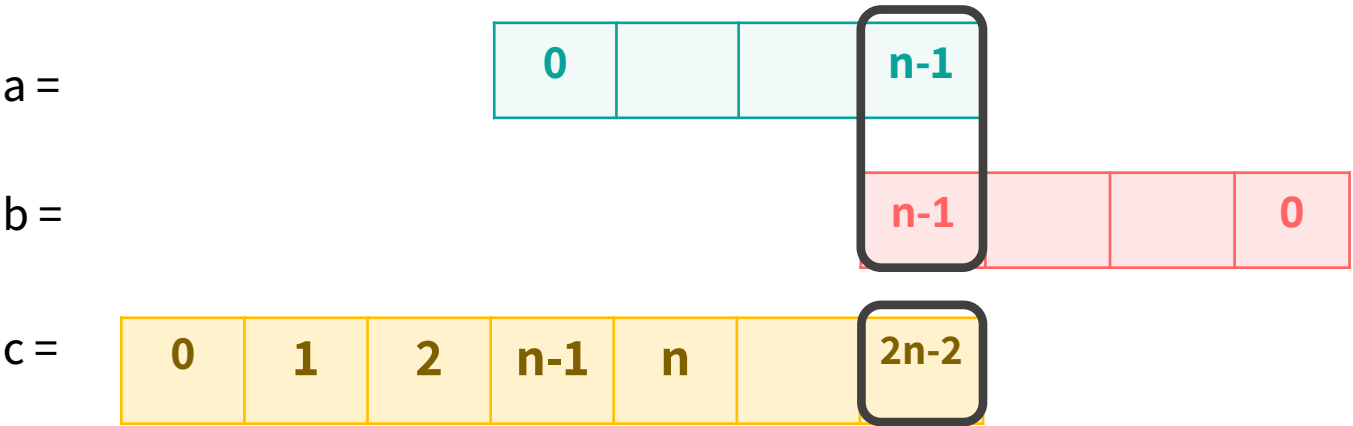
סימון: $c = a \otimes b$

הגדרה מתמטית: סכום מכפלות הזוגות באינדקסים המתאימים ב a ו- b שווה לאיבר באינדקס אחד בוקטור c

כאשר: $c_j = \sum_{k=0}^j a_k \cdot b_{j-k}$ (סכום האינדקסים של הרכיבים שווה לאינדקס של c)

משמעות גאומטרית:

הזזת וקטור אחד הפוך על פני וקטור שני, כאשר לכל הזזה כזו מותאם ערך מספרי שהוא סכום המכפלות נקודה-נקודה.



תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

הרעיון

- בהתאמת מחרוזות, בכל שלב אנו משווים את התבנית P , שהיא באורך m , ל"חלון" באורך m בטקסט T .
 - אם נתייחס לטקסט ולתבנית בתור שני וקטורים, הרי שהקונבולוציה $T \otimes P$ גם היא מזיזה את התבנית לאורך הטקסט, ובכל שלב מעמידה את m התווים של התבנית מול m תווים של הטקסט.
- נוכל לנצל תכונה זו כדי לבצע התאמת מחרוזות באמצעות קונבולוציה.

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה - התאמת הקלט

דוגמה:

T=

1	1	0	1	0	0	1	1	0
---	---	---	---	---	---	---	---	---

P=

1	1	0
---	---	---

הגדרות הקונבולוציה	התאמת הקלט
הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>	
הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>	

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה - התאמת הקלט

הגדרות הקונבולוציה	התאמת הקלט
הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>	<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית
הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>	

דוגמה:

T=

1	1	0	1	0	1	1	1	0
---	---	---	---	---	---	---	---	---

P=

1	1	0
---	---	---

P'=

0	1	1
---	---	---

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה - התאמת הקלט

דוגמה:

T=	1	1	0	1	0	1	1	1	0
P=	1	1	0						
P'=	0	1	1	0	0	0	0	0	0

הגדרות הקונבולוציה	התאמת הקלט
הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>	<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית
הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>	נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $p'[k] = p_{m-1-k}$ $m \leq k < n$ $p'[k] = 0$

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

הקונבולוציה תהפוך בחזרה את התבנית ונקבל:

דוגמה:

T=	1	1	0	1	0	1	1	1	0
P'=	0	0	0	0	0	0	1	1	0
	$p_{n-1} \dots p_{m-1}$					p_1	p_0		

התאמת הקלט	הגדרות הקונבולוציה
<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית	הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>
נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m \quad p'[k] = p_{m-1-k}$ $m \leq k < n \quad p'[k] = 0$	הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

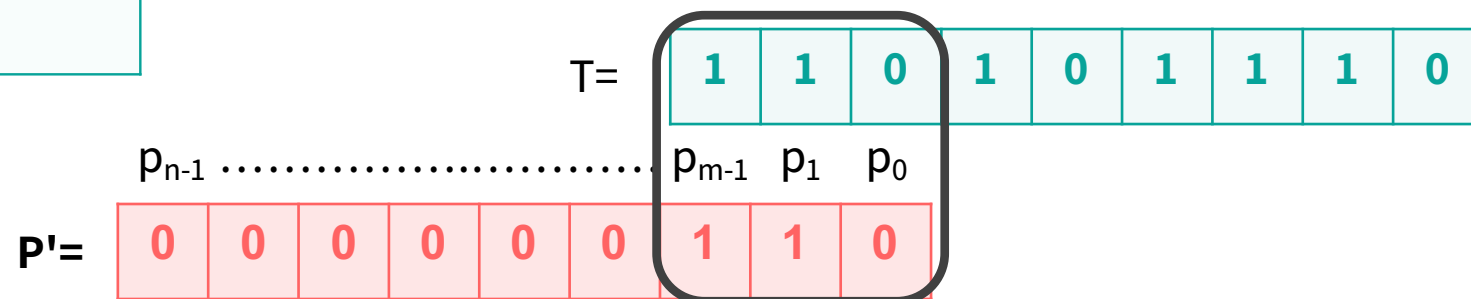
בניית הפתרון - רדוקציה לקונבולוציה

הגדרות הקונבולוציה	התאמת הקלט
הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>	<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית
הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>	נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $p'[k] = p_{m-1-k}$ $m \leq k < n$ $p'[k] = 0$

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

דוגמה:



עבור $m-1 \leq j \leq n-1$ יהיו בדיוק m זוגות של תווים שעומדים זה מול זה.

כאשר $j = m-1$, כל התווים של התבנית P עומדים מול m התווים הראשונים של הטקסט T

$$c_j = c_{m-1} = \sum_{k=0}^{m-1} T_k \cdot p'_{m-1-k} = \sum_{k=0}^{m-1} T_k \cdot p_{m-1-(m-1-k)} = \sum_{k=0}^{m-1} T_k \cdot p_k$$

תרגיל 1

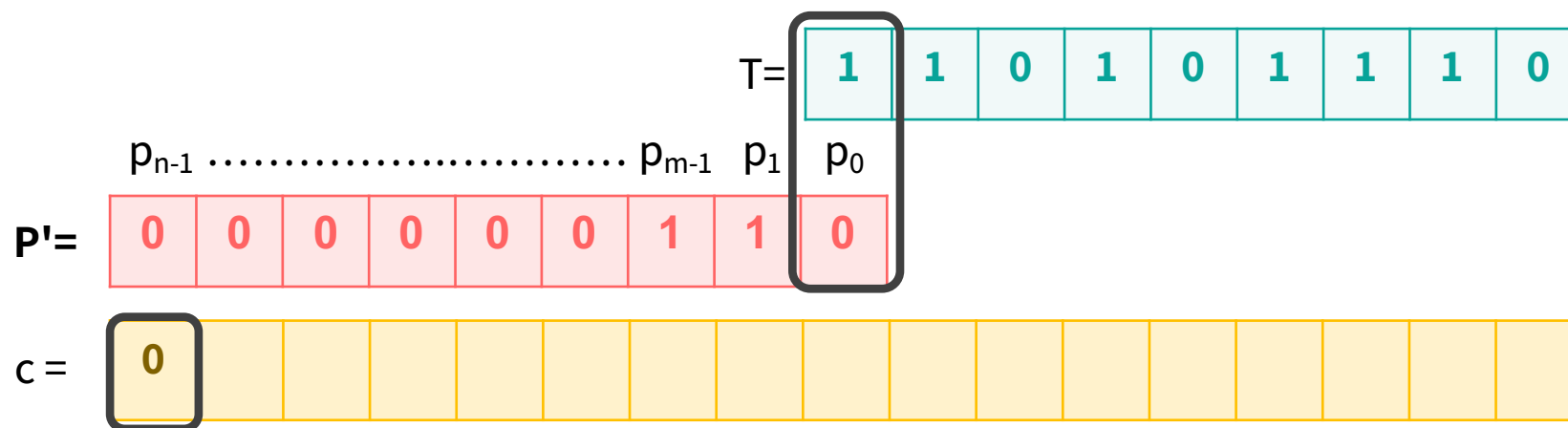
בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

דוגמה:



הגדרות הקונבולוציה	התאמת הקלט
הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>	<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית
הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>	נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $p'[k] = p_{m-1-k}$ $m \leq k < n$ $p'[k] = 0$

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

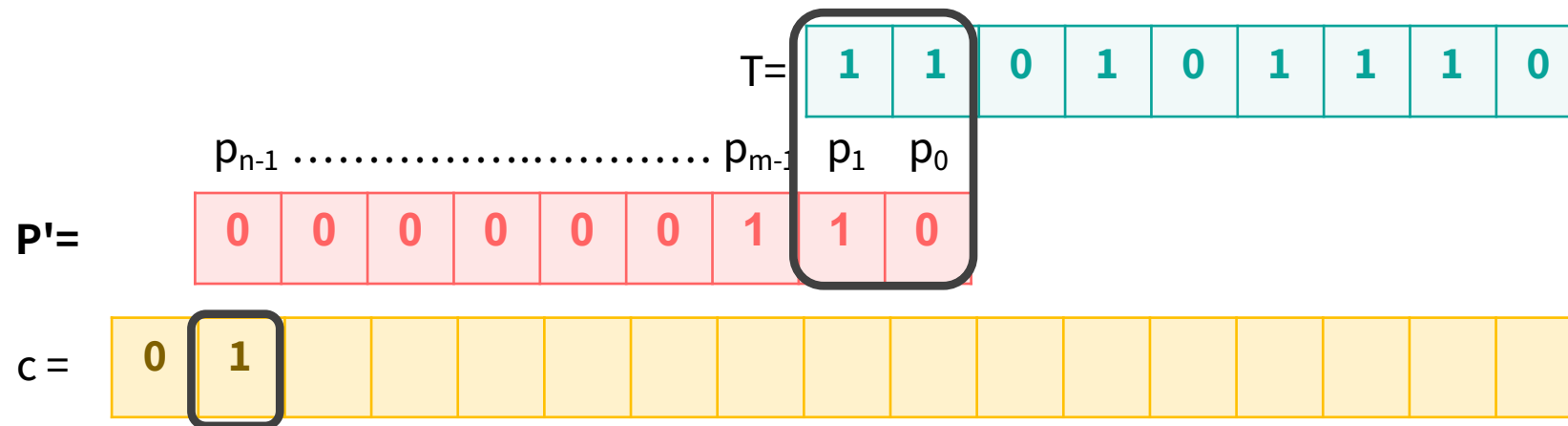
בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

דוגמה:

התאמת הקלט	הגדרות הקונבולוציה
<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית	הקונבולוציה מזיזה את <u>הפוך</u> הקטור השני כשהוא
נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $m \leq k < n$	הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>



תרגיל 1

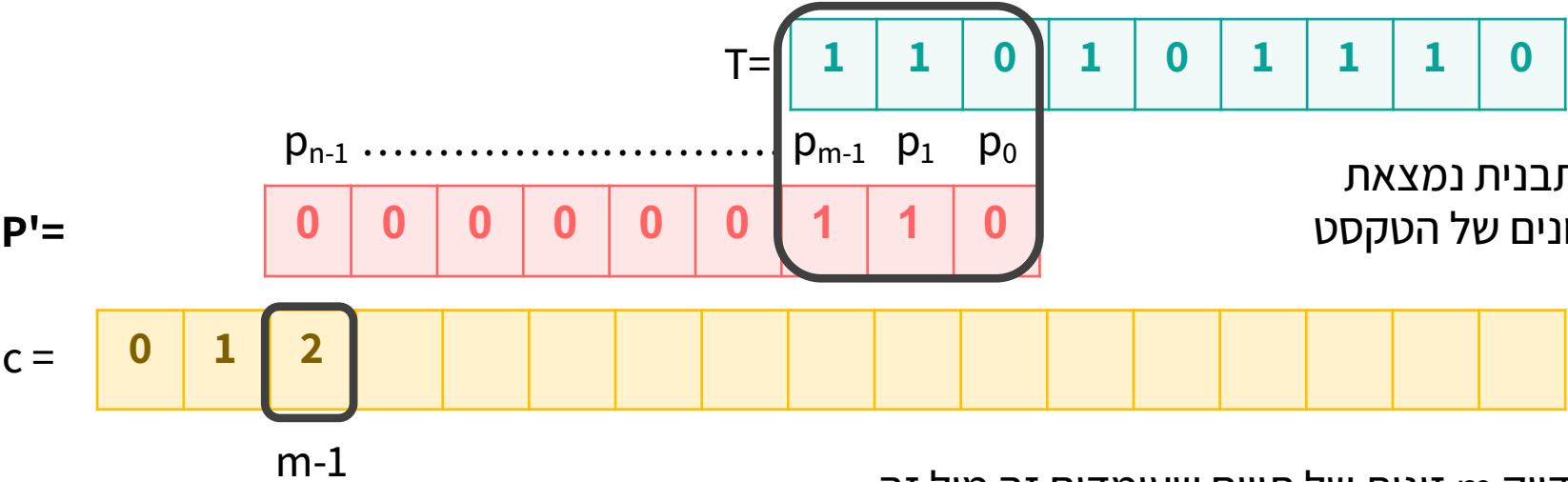
בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

דוגמה:



פעם ראשונה שכל התבנית נמצאת מול m התווים הראשונים של הטקסט

עבור $m-1 \leq j \leq n-1$ יהיו בדיוק m זוגות של תווים שעומדים זה מול זה.

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $P \otimes T$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

התאמת הקלט	הגדרות הקונבולוציה
<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית	הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>
נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $m \leq k < n$	הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>

ההתאמה האחרונה של התבנית עם m התווים האחרונים של הטקסט

P'≡

$c =$

0	1	2	1	1	1	0	2	2								
---	---	---	---	---	---	---	---	---	--	--	--	--	--	--	--	--

$m-1$ $n-1$

דוגמה:

[illegible]

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון - רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

$$c_j = \sum_{k=0}^j T_k \cdot p'_{j-k}$$

עבור $m-1 \leq j \leq n-1$ יהיו בדיוק m זוגות של תווים שעומדים זה מול זה.

דוגמה:

$T =$

1	1	0	1	0	1	1	1	0
p_{n-1}		p_{m-1}	p_1	p_0				
0	0	0	0	0	0	1	1	0

$P' =$

$c =$

0	1	2	1	1	1	0	2	2									
		$m-1$						$n-1$									

התאים ב- c שרלוונטיים עבור מציאת התאמה

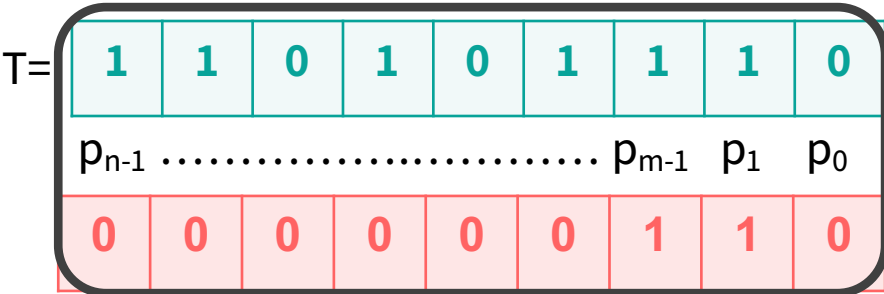
תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

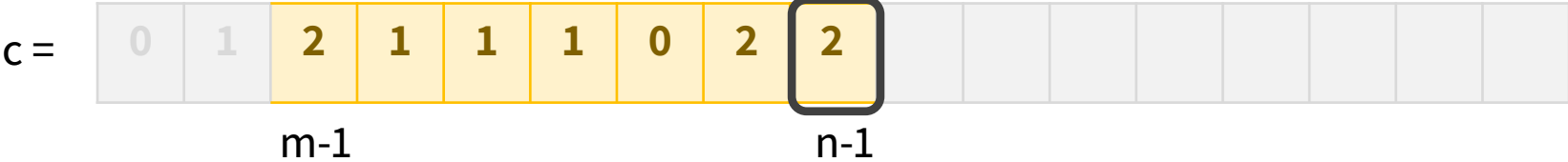
בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

דוגמה:



$P' =$



אפשר לראות שהערך שהתקבל בקונבולוציה במקום c_i הוא מספר ההתאמות של התו 1 בהזזה ה- i

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

דוגמה:

$$T = \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 1 & 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ \hline \end{array}$$

$p_{n-1} \dots p_{m-1} \quad p_1 \quad p_0$

$$P' = \begin{array}{|c|c|c|c|c|c|c|c|c|} \hline 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ \hline \end{array}$$

$P' =$

$$c = \begin{array}{|c|} \hline 0 & 1 & 2 & 1 & 1 & 1 & 0 & 2 & 2 & & & & & & & & & & & \\ \hline \end{array}$$

$m-1 \qquad \qquad \qquad n-1$

אפשר לראות שהערך שהתקבל בקונבולוציה במקום ה- c הוא מספר ההתאמות של התו 1 בהזזה ה- i

איך נוכל למצוא את מספר ההתאמות של התו 0?

תרגיל 1

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

בניית הפתרון – רדוקציה לקונבולוציה

מה נקבל אם נבצע את הקונבולוציה $T \otimes P'$?

אפשר לראות שהערך שהתקבל בקונבולוציה במקום i הוא מספר ההתאמות של התו 1 בהזזה i

התאמת הקלט	הגדרות הקונבולוציה
<u>נהפוך את התבנית</u> כדי לקבל את ההתאמות למחרוזת המקורית	הקונבולוציה מזיזה את הוקטור השני כשהוא <u>הפוך</u>
נרפד את התבנית P באפסים (כדי שהם לא ישפיעו על הסכום) $0 \leq k < m$ $p'[k] = p_{m-1-k}$ $m \leq k < n$ $p'[k] = 0$	הקונבולוציה מוגדרת על שני מערכים <u>באותו אורך</u>

$T =$

1	1	0	1	0	1	1	1	0
p_{n-1}		p_{m-1}	p_1	p_0				
0	0	0	0	0	0	1	1	0

$P' =$

$c =$

0	1	2	1	1	1	0	2	2									
		$m-1$						$n-1$									

איך נוכל למצוא את מספר ההתאמות של התו 0?

נבצע פעולת NOT על הקלט והתבנית ונבצע מחדש את הקונבולוציה.

תרגיל 1-פתרון

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

לסיכום:

נבצע את **הרדוקציה** לקונבולוציה ונפתור:

1. **נהפוך את התבנית P**
2. **נרפד את סוף התבנית באפסים עד לאורך הטקסט T**
3. **נבצע קונבולוציה כדי למצוא את ההתאמות לתו 1**
4. נבצע NOT על הטקסט T והתבנית P
5. **נחזור על הפעולות 1-3 כדי למצוא את ההתאמה לתו 0**
6. נחבר את שני הוקטורים ובכל תא שמספר ההתאמות זהה לאורך התבנית – נדפיס את ההתאמה.

convolution_string_matcher(T,P)

1. $P' \leftarrow \text{Reverse}(P)$
2. $P' \leftarrow \text{PadWithZero}(P', \text{length}[T])$
3. $c_1 \leftarrow \text{convulotion}(T, P')$
4. $\bar{P} \leftarrow \text{NOT}(P)$
5. $\bar{T} \leftarrow \text{NOT}(T)$
6. $\bar{P}' \leftarrow \text{Reverse}(\bar{P})$
5. $\bar{P}' \leftarrow \text{PadWithZero}(\bar{P}', \text{length}[\bar{T}])$
6. $c_0 \leftarrow \text{convulotion}(\bar{T}, \bar{P}')$
7. **for** $i \leftarrow m-1$ **to** $\text{length}[c_1]$
8. **do if** $c_1[i] + c_0[i] = \text{length}[P]$
9. **then print** "Pattern occurs with shift" $i - m + 1$


$$i - (m-1) = i - m + 1$$

תזכורת



כאשר $i=m-1$, כל התווים של התבנית P עומדים לראשונה מול m התווים **הראשונים** של הטקסט T לכן, נחסר מהאינדקס בו נמצא התאמה " $m-1$ " כי עד אינדקס זה – כל ההתאמות לא רלוונטיות לפתרון

תרגיל 1-פתרון

בהינתן טקסט T ותבנית P מעל אלפבית בינארי, הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

לסיכום:

נבצע את **הרדוקציה** לקונבולוציה ונפתור:

1. **נהפוך את התבנית P**
2. **נרפד את סוף התבנית באפסים עד לאורך הטקסט T**
3. **נבצע קונבולוציה כדי למצוא את ההתאמות לתו 1**
4. נבצע NOT על הטקסט T והתבנית P
5. **נחזור על הפעולות 1-3 כדי למצוא את ההתאמה לתו 0**
6. נחבר את שני הוקטורים ובכל תא שמספר ההתאמות זהה לאורך התבנית – נדפיס את ההתאמה.

convolution_string_matcher(T,P)

1. $P' \leftarrow \text{Reverse}(P)$
2. $P' \leftarrow \text{PadWithZero}(P', \text{length}[T])$
3. $c_1 \leftarrow \text{convulotion}(T, P')$
4. $\bar{P} \leftarrow \text{NOT}(P)$
5. $\bar{T} \leftarrow \text{NOT}(T)$
6. $\bar{P}' \leftarrow \text{Reverse}(\bar{P})$
7. $\bar{P}' \leftarrow \text{PadWithZero}(\bar{P}', \text{length}[\bar{T}])$
8. $c_0 \leftarrow \text{convulotion}(\bar{T}, \bar{P}')$
9. **for** $i \leftarrow m-1$ **to** $\text{length}[c_1]$
 do if $c_1[i] + c_0[i] = \text{length}[P]$
 then print "Pattern occurs with shift" $i - m + 1$

ס"כ עלות: $O(n \log n)$

**בהרצאה נראה איך אפשר להשתמש באלגוריתם FFT
לפעולת הקונבולוציה בעלות $O(n \log n)$*

$$i - (m-1) = i - m + 1$$

תזכורת



כאשר $i=m-1$, כל התווים של התבנית P עומדים לראשונה מול m התווים **הראשונים** של הטקסט T לכן, נחסר מהאינדקס בו נמצא התאמה " $m-1$ " כי עד אינדקס זה – כל ההתאמות לא רלוונטיות לפתרון

תרגיל 1-הכללה

בהינתן טקסט T ותבנית P מעל אלפבית בגודל R , הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

תרגיל 1-הכללה

בהינתן טקסט T ותבנית P מעל אלפבית בגודל R , הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

פתרון

בכל שלב נבחר אות אחת, נסמן אותה בתור "1" ואת שאר האותיות בתור "0" ונבדוק בעזרת הרדוקציה לקונבולוציה את מספר ההתאמות.

נחזור על התהליך עבור כל אחת מהאותיות, ס"כ R קונבולוציות.

לסיום, נסכום את R הוקטורים שהתקבלו – במקום בו הסכום הוא אורך התבנית P , m , יש התאמה מלאה.

תרגיל 1-הכללה

בהינתן טקסט T ותבנית P מעל אלפבית בגודל R , הסבירו כיצד ניתן לבצע התאמת מחרוזות בעזרת קונבולוציה.

פתרון

בכל שלב נבחר אות אחת, נסמן אותה בתור "1" ואת שאר האותיות בתור "0" ונבדוק בעזרת הרדוקציה לקונבולוציה את מספר ההתאמות.

נחזור על התהליך עבור כל אחת מהאותיות, ס"כ R קונבולוציות.

לסיום, נסכום את R הוקטורים שהתקבלו – במקום בו הסכום הוא אורך התבנית P , m , יש התאמה מלאה.

ס"כ עלות: $O(Rn \log n) = O(n \log n)$

* בהרצאה נראה איך אפשר להשתמש באלגוריתם FFT

לפעולת הקונבולוציה בעלות $O(n \log n)$